

Scaffold Module Documentation

Business-style build record for the initial monorepo setup, including API, Web, Worker, commands, validation outputs, and completion status.

TIME BLOCK

10:00 AM - 10:45 AM

MODULE

Scaffold

STATUS

Complete

Objective

Set up the base monorepo structure for the Email Campaign Platform and confirm that the API, Web, and Worker apps can typecheck, build, and run independently.

Project Structure

```
email-campaign-platform/  
  apps/  
    api/  
      src/index.ts  
      package.json  
      tsconfig.json  
    web/  
      src/main.tsx  
      index.html  
      package.json  
      tsconfig.json  
      vite.config.ts  
    worker/  
      src/index.ts  
      package.json  
      tsconfig.json  
  prisma/schema.prisma  
  .env.example  
  package.json  
  pnpm-workspace.yaml
```

API App

Backend REST API using Node.js, Express, TypeScript, and future Prisma integration.

Web App

React and Vite frontend for admin screens and future public subscriber screens.

Worker App

Background worker for scheduled sends, imports, AWS SQS processing, and SES events.

Root Workspace Setup

The root workspace controls all package-level commands.

```
{
  "dev": "pnpm -r --parallel dev",
  "build": "pnpm -r build",
  "lint": "pnpm -r lint",
  "typecheck": "pnpm -r typecheck",
  "db:generate": "pnpm --filter api prisma:generate",
  "db:migrate": "pnpm --filter api prisma:migrate",
  "db:seed": "pnpm --filter api prisma:seed",
  "db:studio": "pnpm --filter api prisma:studio"
}
```

```
packages:
  - "apps/*"
  - "packages/*"
```

API Scaffold

Item	Details
Location	apps/api
Files	package.json, tsconfig.json, src/index.ts
Runtime	Node.js, Express, TypeScript
Purpose	Backend routes, auth, database access, campaign management, tracking, analytics, AWS webhooks.

API Commands

```
pnpm --filter api typecheck
pnpm --filter api build
```

```
pnpm --filter api dev
```

Health Route

```
GET http://localhost:4000/health

{
  "status": "ok",
  "service": "api"
}
```

Note: `http://localhost:4000/` can show Cannot GET / at scaffold stage because only `/health` exists.

Web Scaffold

Item	Details
Location	apps/web
Files	package.json, tsconfig.json, vite.config.ts, index.html, src/main.tsx
Runtime	React, Vite, TypeScript
Purpose	Admin dashboard and public subscriber-facing pages.

Web Commands

```
pnpm --filter web typecheck
pnpm --filter web build
pnpm --filter web dev
```

Successful Build Output

```
vite v6.4.2 building for production...
27 modules transformed.
dist/index.html                0.35 kB
dist/assets/index-DVZ-duwc.js 194.77 kB
built in 779ms
```

Expected browser URL: `http://localhost:5173/`

Worker Scaffold

Item	Details
------	---------

Location	apps/worker
Files	package.json, tsconfig.json, src/index.ts
Runtime	Node.js, TypeScript
Purpose	Scheduled sending, CSV imports, SQS jobs, SES event processing, bounce and complaint handling.

Worker Commands

```
pnpm --filter worker typecheck
pnpm --filter worker build
pnpm --filter worker dev
```

Expected Output

```
Worker scaffold running.
Worker heartbeat
```

Issues Found and Resolved

Empty API package file: fixed [ERR_PNPM_JSON_PARSE] by adding a valid apps/api/package.json.

Empty Web package file: fixed [ERR_PNPM_JSON_PARSE] by adding a valid apps/web/package.json.

Invalid Vite config: fixed config must export or return an object by exporting defineConfig.

Final Verification

Root Typecheck

```
pnpm typecheck

apps/api typecheck completed
apps/web typecheck completed
apps/worker typecheck completed
```

Root Build

```
pnpm build
```

```
apps/web build completed  
apps/api build completed  
apps/worker build completed
```

Completion Status

Scaffold Module Complete

Root workspace configured, API app scaffolded, Web app scaffolded, Worker app scaffolded, TypeScript configured, Vite configured, Express server configured, worker process configured, all apps typecheck successfully, and all apps build successfully.

Code Appendix

This appendix contains the complete scaffold code used in the module.

Root: package.json

```
{  
  "name": "email-campaign-platform",  
  "version": "1.0.0",  
  "private": true,  
  "description": "In-house email campaign platform",  
  "type": "module",  
  "packageManager": "pnpm@11.0.9",  
  "workspaces": [  
    "apps/*",  
    "packages/*"  
  ],  
  "scripts": {  
    "dev": "pnpm -r --parallel dev",  
    "build": "pnpm -r build",  
    "lint": "pnpm -r lint",  
    "typecheck": "pnpm -r typecheck",  
    "clean": "node -e \"const fs=require('fs'); for (const p of ['node_modules','turbo'])  
fs.rmSync(p,{recursive:true,force:true});\"",  
    "db:generate": "pnpm --filter api prisma:generate",  
    "db:migrate": "pnpm --filter api prisma:migrate",  
    "db:seed": "pnpm --filter api prisma:seed",  
    "db:studio": "pnpm --filter api prisma:studio"  
  }  
}
```

Root: pnpm-workspace.yaml

```
packages:
  - "apps/*"
  - "packages/*"
allowBuilds:
  '@prisma/client': true
  '@prisma/engines': true
esbuild: true
prisma: true
```

API: apps/api/package.json

```
{
  "name": "api",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "main": "dist/index.js",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc -p tsconfig.json",
    "start": "node dist/index.js",
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "lint": "echo \"lint api\"",
    "prisma:generate": "prisma generate --schema ../../prisma/schema.prisma",
    "prisma:migrate": "prisma migrate dev --schema ../../prisma/schema.prisma",
    "prisma:seed": "tsx src/seed.ts",
    "prisma:studio": "prisma studio --schema ../../prisma/schema.prisma"
  },
  "dependencies": {
    "@prisma/client": "^6.7.0",
    "bcryptjs": "^3.0.2",
    "cors": "^2.8.5",
    "dotenv": "^16.5.0",
    "express": "^5.1.0",
    "jsonwebtoken": "^9.0.2",
    "zod": "^3.24.4"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.6",
    "@types/cors": "^2.8.17",
    "@types/express": "^5.0.1",
    "@types/jsonwebtoken": "^9.0.9",
    "@types/node": "^22.15.17",
    "prisma": "^6.7.0",
    "tsx": "^4.19.4",
    "typescript": "^5.8.3"
  }
}
```

API: apps/api/tsconfig.json

```

{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "types": ["node"]
  },
  "include": ["src"]
}

```

API: apps/api/src/index.ts

```

import "dotenv/config";
import express from "express";
import cors from "cors";

const app = express();

const port = Number(process.env.API_PORT ?? 4000);
const webOrigin = process.env.WEB_ORIGIN ?? "http://localhost:5173";

app.use(cors({ origin: webOrigin }));
app.use(express.json());

app.get("/", (_req, res) => {
  res.json({
    message: "Email Campaign Platform API",
    health: "/health"
  });
});

app.get("/health", (_req, res) => {
  res.json({
    status: "ok",
    service: "api"
  });
});

app.listen(port, () => {
  console.log(`API running on http://localhost:${port}`);
});

```

Web: apps/web/package.json

```
{
  "name": "web",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0 --port 5173",
    "build": "tsc -p tsconfig.json && vite build",
    "preview": "vite preview --host 0.0.0.0 --port 5173",
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "lint": "echo \"lint web\""
  },
  "dependencies": {
    "@vitejs/plugin-react": "^4.4.1",
    "vite": "^6.3.5",
    "typescript": "^5.8.3",
    "react": "^19.1.0",
    "react-dom": "^19.1.0"
  },
  "devDependencies": {
    "@types/react": "^19.1.3",
    "@types/react-dom": "^19.1.3"
  }
}
```

Web: apps/web/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "jsx": "react-jsx",
    "strict": true,
    "skipLibCheck": true,
    "noEmit": true,
    "allowImportingTsExtensions": true
  },
  "include": ["src", "vite.config.ts"]
}
```

Web: apps/web/vite.config.ts

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()]
});
```


Web: apps/web/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Email Campaign Platform</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Web: apps/web/src/main.tsx

```
import React from "react";
import { createRoot } from "react-dom/client";

function App() {
  return (
    <main>
      <h1>Email Campaign Platform</h1>
      <p>Web app scaffold is running.</p>
    </main>
  );
}

createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Worker: apps/worker/package.json

```
{
  "name": "worker",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "main": "dist/index.js",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc -p tsconfig.json",
    "start": "node dist/index.js",
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "lint": "echo \"lint worker\""
  },
  "dependencies": {
    "@prisma/client": "^6.7.0",
  }
}
```

```
"dotenv": "^16.5.0"
},
"devDependencies": {
  "@types/node": "^22.15.17",
  "tsx": "^4.19.4",
  "typescript": "^5.8.3"
}
}
```

Worker: apps/worker/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "types": ["node"]
  },
  "include": ["src"]
}
```

Worker: apps/worker/src/index.ts

```
import "dotenv/config";

console.log("Worker scaffold running.");

setInterval(() => {
  console.log("Worker heartbeat");
}, 30_000);
```

Next Module

Database Module: finalize Prisma schema, generate Prisma client, create database migration, add seed script, seed organisation, seed super admin user, seed five starter templates, seed sample contact list, and seed sample contacts.